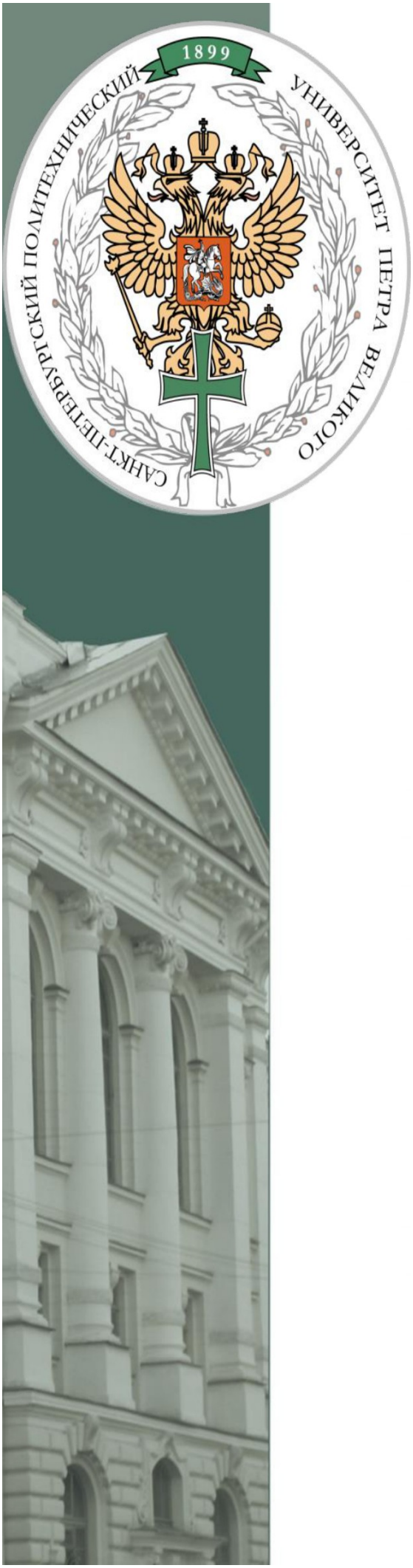


A

EXAM PRACTICAL REPORT

ON

Organizing Colors with MiniSom

WRITTEN BY

MOHAMMAD AZIMI

Group No. 5140901/61701

ON

INTELLIGENT SYSTEMS

October 17, 2026

INTRODUCTION

MiniSom is a Python library for Self-Organizing Maps (SOMs), a form of unsupervised neural network. A SOM represents input data using a two-dimensional grid of learned prototypes. Similar inputs tend to match nearby cells, which makes relationships in the data easier to inspect. [1]

This practical work uses MiniSom to organize RGB colors. The experiment trains a 16 by 16 map on 1,000 generated colors and evaluates it on 500 separate colors. The resulting map contains smoother color regions, and its topographic error decreases from 97.6% to 9.2%.

Install the Library and Prepare RGB Data

```
python -m pip install minisom==2.3.5 numpy matplotlib
```

Each color is a vector containing red, green and blue values between 0 and 1. The dataset has no class labels. A fixed seed makes this demonstration reproducible.

```
import numpy as np
from minisom import MiniSom

rng = np.random.default_rng(42)
colors = rng.random((1000, 3))
test_colors = rng.random((500, 3))
```

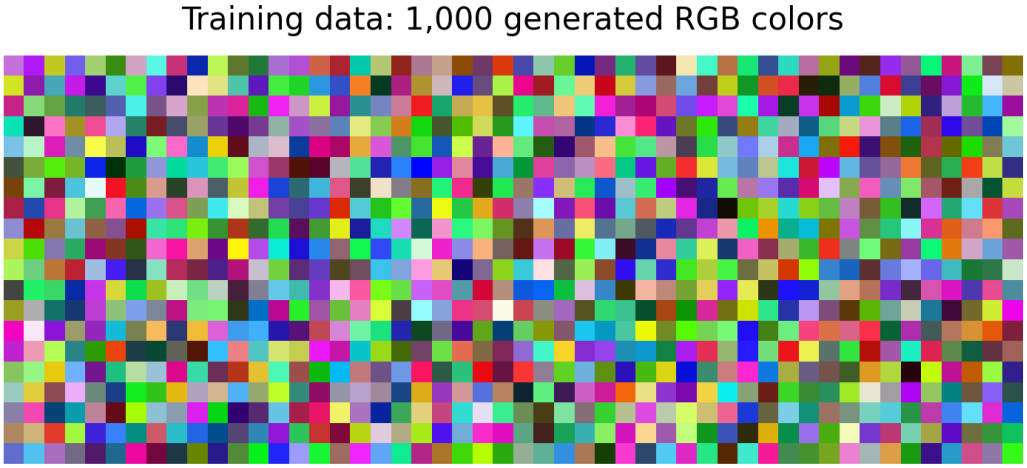


Figure 1 The 1,000 RGB vectors used for training

The 500 evaluation colors remain separate from the training data. No dataset download or GPU is required for this example.

Define the Self Organizing Map

The map contains 256 cells. Each cell stores a three-dimensional RGB prototype. The random_weights_init method initializes these prototypes by selecting colors from the training data. [2]

```
som = MiniSom(16, 16, 3, sigma=2.0,
              learning_rate=0.5, random_seed=42)
som.random_weights_init(colors)
before = som.get_weights().copy()
```

Parameter	Value	Meaning
Map size	16 x 16	Number of rows and columns
Input length	3	Red, green and blue features
Sigma	2.0	Initial neighborhood size
Learning rate	0.5	Initial weight update rate
Random seed	42	Reproducible initialization

Learning Procedure

For each update, the model finds the prototype closest to an input color. It moves this prototype and neighboring prototypes toward the input. The neighborhood and learning rate decrease during training, allowing the map to refine its arrangement. [2]

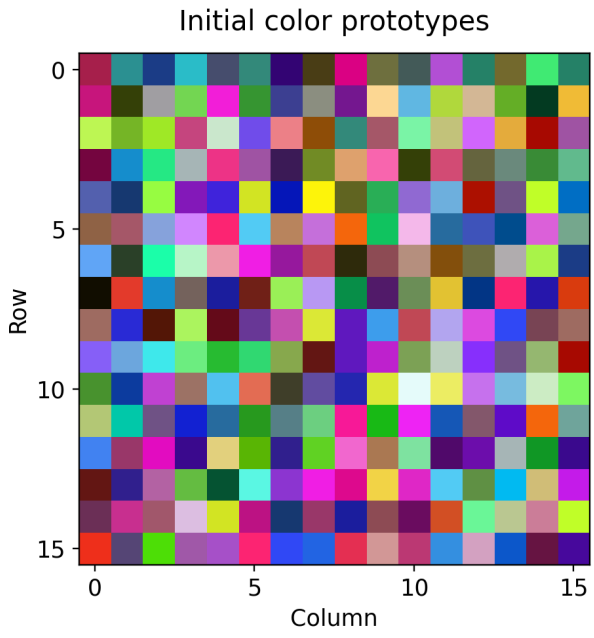


Figure 2 Initial prototypes before training

Train the Map

The train_random method trains the model for 20,000 individual updates. This value counts updates rather than complete passes through the dataset. The final colors come directly from the learned weights.

```
som.train_random(colors, 20000)
after = som.get_weights().copy()
```

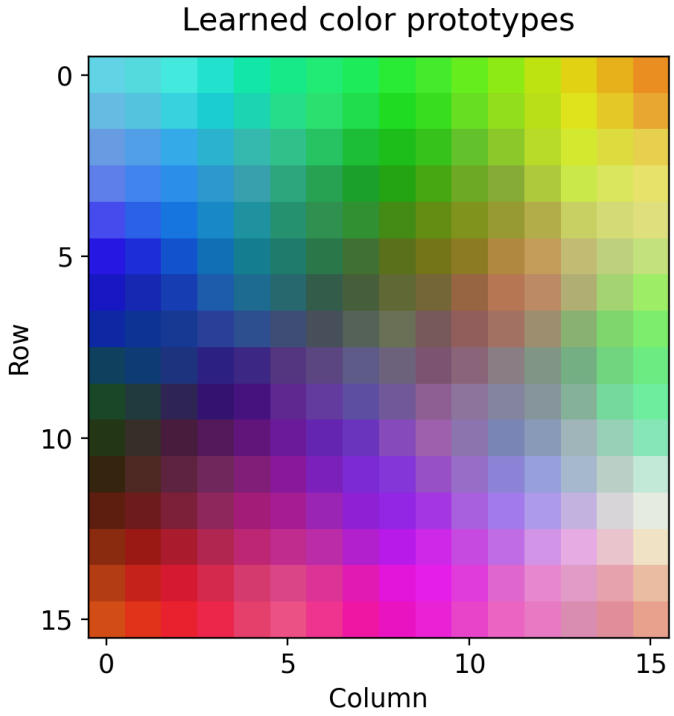


Figure 3 Learned prototypes after 20,000 updates

Locate a New Color

The winner method finds the closest prototype and returns its grid coordinates. For a pure red input, this run returns row 15 and column 2, using zero-based indices.

```
query = np.array([1.0, 0.0, 0.0])
position = som.winner(query)
prototype = som.get_weights()[position]
```

The matching prototype is approximately [0.914, 0.132, 0.185]. It represents a learned color near the red input. The method returns a position rather than a class name.

Plot and Evaluate the Results

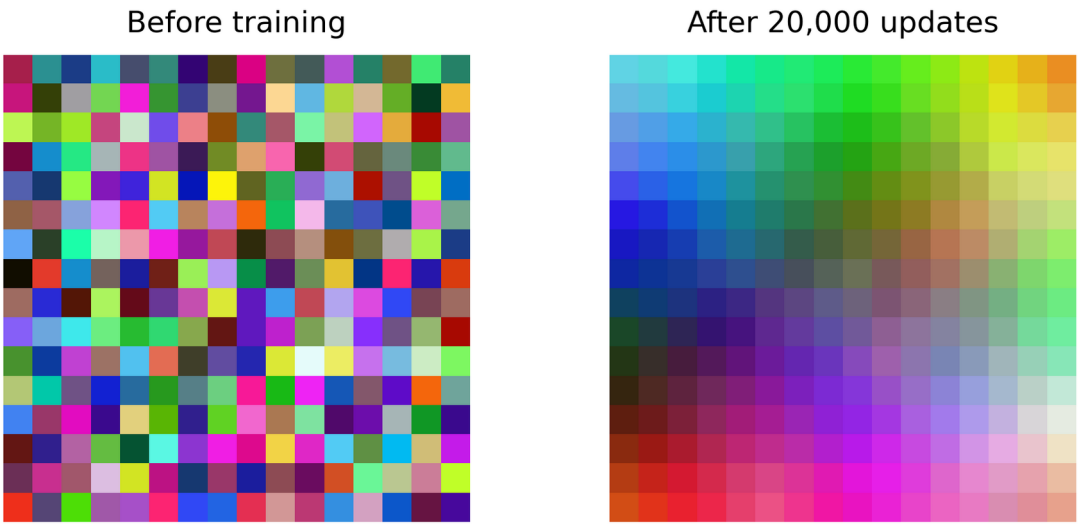


Figure 4 The same model grid before and after training

The initial grid contains randomly selected training colors. After training, nearby cells show smoother transitions. The program does not apply an additional sorting step to the learned colors.

Evaluation on Separate Data

Both evaluations use the same 500 RGB vectors. Topographic error measures neighborhood consistency. Quantization error measures the average Euclidean distance from each input to its closest prototype. Lower values are better for both metrics. [2]

```
topographic = som.topographic_error(test_colors)
quantization = som.quantization_error(test_colors)
```

Metric on 500 evaluation colors	Before	After
Topographic error	97.6%	9.2%
Quantization error	0.096195	0.091848

Topographic error counts an input as an error when its two closest prototypes occupy non-neighboring cells. The decrease shows better organization in this experiment. These values are separate from classification accuracy because the dataset contains no class labels.

Complete Python Example

The following example reproduces the model settings, evaluation metrics and before-and-after figure. Run it in a Python environment with NumPy, Matplotlib and MiniSom installed.

```
import numpy as np
import matplotlib.pyplot as plt
from minisom import MiniSom

rng = np.random.default_rng(42)
colors = rng.random((1000, 3))
test_colors = rng.random((500, 3))

som = MiniSom(16, 16, 3, sigma=2.0,
              learning_rate=0.5, random_seed=42)
som.random_weights_init(colors)
before = som.get_weights().copy()
te_before = som.topographic_error(test_colors)
qe_before = som.quantization_error(test_colors)

som.train_random(colors, 20000)
after = som.get_weights().copy()
te_after = som.topographic_error(test_colors)
qe_after = som.quantization_error(test_colors)

print(f"Topographic: {te_before:.3f} -> {te_after:.3f}")
print(f"Quantization: {qe_before:.6f} -> {qe_after:.6f}")
position = tuple(map(int, som.winner([1.0, 0.0, 0.0])))
print("Red position:", position)

fig, axes = plt.subplots(1, 2, figsize=(10, 5))
for ax, values, title in zip(
        axes, [before, after], ["Before", "After"]):
    ax.imshow(values, interpolation="nearest")
    ax.set_title(title)
    ax.axis("off")
fig.tight_layout()
fig.savefig("minisom_result.png", dpi=180)
```

Recorded Console Output

```
Topographic: 0.976 -> 0.092
Quantization: 0.096195 -> 0.091848
Red position: (15, 2)
```

Discussion and Conclusion

MiniSom learns representative colors from numerical input. The experiment shows that a small unsupervised model can produce a useful visual arrangement without class labels. The map contains learned prototypes, so its cells do not have to be exact copies of the original colors.

Applications

SOMs can help visualize similarities between samples and explore groups in data. In an image-processing workflow, their learned prototypes can also form a representative color palette. For data with more features, the grid provides a compact way to inspect relationships. [1]

Limitations

Feature scaling affects which samples appear similar. In this example, all RGB features already share the same range. Other datasets may require normalization. Map dimensions, neighborhood settings and training duration also influence the result.

The experiment uses synthetic RGB data and one random seed. Its numerical results describe this demonstration rather than a general benchmark. A different seed can change the orientation or arrangement. Predicting named classes requires an additional labeling step.

Conclusion

The trained 16 by 16 map organizes similar colors into smoother regions. On the 500 evaluation colors, topographic error decreases from 97.6% to 9.2%, while quantization error decreases slightly from 0.096195 to 0.091848. MiniSom provides a compact way to demonstrate unsupervised learning in Python.

References

[1] MiniSom official repository and README. https://github.com/JustGlowing/minisom

[2] MiniSom source code and evaluation methods.
https://github.com/JustGlowing/minisom/blob/master/minisom.py

Experimental outputs: minisom_colors_demo.py, results.json and color_map.npz. MiniSom 2.3.5, NumPy 2.3.5, Matplotlib 3.10.8 and Python 3.12.14. Run date: October 17, 2026.